
Batch 5 — PHP Backend Security + Audit Layer

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:06

Batch 5 — PHP Backend Security + Audit Layer

Meat Lovers CIMS powered by YohPal

5A.

backend/app/Support/Auth.php

```
<?php

declare(strict_types=1);

namespace Support;

class Auth
{
    public static function user(): ?array
    {
        $headers = getallheaders();
        $authHeader = $headers['Authorization'] ?? $headers['authorization'] ?? null;

        if (!$authHeader || !str_starts_with($authHeader, 'Bearer ')) {
            return null;
        }

        $token = trim(str_replace('Bearer ', '', $authHeader));
        $decoded = base64_decode($token);

        if (!$decoded || !str_contains($decoded, '|')) {
            return null;
        }

        [$userId] = explode('|', $decoded);

        $stmt = Database::connection()->prepare(
            'SELECT id, full_name, email, phone, role, is_active
            FROM users
            WHERE id = :id AND is_active = 1
            LIMIT 1'
        );

        $stmt->execute(['id' => $userId]);
        $user = $stmt->fetch();

        return $user ?: null;
    }

    public static function requireAuth(): array
    {
        $user = self::user();

        if (!$user) {
            Response::error('Authentication required', 401);
        }

        return $user;
    }

    public static function requireRole(array $allowedRoles): array
    {
        $user = self::requireAuth();

        if (!in_array($user['role'], $allowedRoles, true)) {
            Response::error('Permission denied', 403);
        }

        return $user;
    }
}
```

5B.

backend/app/Support/AuditLogger.php

```
<?php

declare(strict_types=1);

namespace Support;

class AuditLogger
{
    public static function log(
        ?int $userId,
        string $module,
        string $action,
        ?int $entityId = null,
        ?array $oldData = null,
        ?array $newData = null
    ): void {
        $stmt = Database::connection()->prepare(
            'INSERT INTO audit_logs
            (user_id, module_name, action_name, entity_id, old_data, new_data, ip_address)
            VALUES (:user_id, :module, :action, :entity_id, :old_data, :new_data, :ip)'
        );

        $stmt->execute([
            'user_id' => $userId,
            'module' => $module,
            'action' => $action,
            'entity_id' => $entityId,
            'old_data' => $oldData ? json_encode($oldData) : null,
            'new_data' => $newData ? json_encode($newData) : null,
            'ip' => $_SERVER['REMOTE_ADDR'] ?? null,
        ]);
    }
}
```

5C.

database/migrations/018_approval_requests.sql

```
CREATE TABLE approval_requests (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    requested_by BIGINT NOT NULL,

    approval_type ENUM(
        'ORDER_CANCELLATION',
        'DISCOUNT',
        'STOCK_ADJUSTMENT',
        'REFUND'
    ) NOT NULL,

    reference_id BIGINT,

    request_reason TEXT,

    request_data JSON,

    approval_status ENUM(
        'PENDING',
        'APPROVED',
        'REJECTED'
    ) DEFAULT 'PENDING',

    approved_by BIGINT,

    approval_notes TEXT,

    approved_at TIMESTAMP NULL,

    FOREIGN KEY (requested_by)
        REFERENCES users(id),

    FOREIGN KEY (approved_by)
        REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

5D.

backend/app/Support/ApprovalEnforcer.php

```
<?php

declare(strict_types=1);

namespace Support;

class ApprovalEnforcer
{
    public static function createRequest(
        int $requestedBy,
        string $type,
        ?int $referenceId,
        string $reason,
        array $requestData = []
    ): int {
        $stmt = Database::connection()->prepare(
            'INSERT INTO approval_requests
            (requested_by, approval_type, reference_id, request_reason, request_data, approval_status)
            VALUES (:requested_by, :type, :reference_id, :reason, :request_data, "PENDING")'
        );

        $stmt->execute([
            'requested_by' => $requestedBy,
            'type' => $type,
            'reference_id' => $referenceId,
            'reason' => $reason,
            'request_data' => json_encode($requestData),
        ]);

        return (int) Database::connection()->lastInsertId();
    }

    public static function requireApproved(string $type, int $referenceId): void
    {
        $stmt = Database::connection()->prepare(
            'SELECT id FROM approval_requests
            WHERE approval_type = :type
            AND reference_id = :reference_id
            AND approval_status = "APPROVED"
            LIMIT 1'
        );

        $stmt->execute([
            'type' => $type,
            'reference_id' => $referenceId,
        ]);

        if (!$stmt->fetch()) {
            Response::error('Approved request required before this action can proceed', 403);
        }
    }
}
```

5E.

backend/app/Controllers/ApprovalsController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Auth;
use Support\Database;
use Support\Request;
use Support\Response;
use Support\AuditLogger;

class ApprovalsController
{
    public function index(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

        $stmt = Database::connection()->query(
            'SELECT ar.*, u.full_name AS requested_by_name
            FROM approval_requests ar
            JOIN users u ON u.id = ar.requested_by'
        );
    }
}
```

```

        ORDER BY ar.created_at DESC'
    );

    Response::success(['approvals' => $stmt->fetchAll()]);
}

public function approve(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE approval_requests
        SET approval_status = "APPROVED",
            approved_by = :approved_by,
            approval_notes = :notes,
            approved_at = NOW()
        WHERE id = :id'
    );

    $stmt->execute([
        'approved_by' => $user['id'],
        'notes' => $data['approval_notes'] ?? null,
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'APPROVALS',
        'APPROVE_REQUEST',
        $id,
        null,
        $data
    );

    Response::success([], 'Approval request approved');
}

public function reject(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'UPDATE approval_requests
        SET approval_status = "REJECTED",
            approved_by = :approved_by,
            approval_notes = :notes,
            approved_at = NOW()
        WHERE id = :id'
    );

    $stmt->execute([
        'approved_by' => $user['id'],
        'notes' => $data['approval_notes'] ?? null,
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'APPROVALS',
        'REJECT_REQUEST',
        $id,
        null,
        $data
    );

    Response::success([], 'Approval request rejected');
}
}

```

5F. Update

backend/routes/api.php

Add these imports:

```
use Controllers\ApprovalsController;
```

Add these routes:

```

$router->get('/api/approvals', [ApprovalsController::class, 'index']);
$router->post('/api/approvals/{id}/approve', [ApprovalsController::class, 'approve']);
$router->post('/api/approvals/{id}/reject', [ApprovalsController::class, 'reject']);

```

```
$router->post('/api/orders/{id}/request-cancellation', [OrdersController::class, 'requestCancellation']);
$router->post('/api/orders/{id}/request-discount', [OrdersController::class, 'requestDiscount']);

$router->post('/api/inventory/request-stock-adjustment', [InventoryController::class,
'requestStockAdjustment']);
```

5G. Update

backend/app/Controllers/InventoryController.php

Add these imports:

```
use Support\Auth;
use Support\AuditLogger;
use Support\ApprovalEnforcer;
```

Add this method inside the class:

```
public function requestStockAdjustment(): void
{
    $user = Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'STOREKEEPER'
    ]);

    $data = Request::body();

    $approvalId = ApprovalEnforcer::createRequest(
        (int) $user['id'],
        'STOCK_ADJUSTMENT',
        (int) $data['product_id'],
        $data['reason'] ?? 'Stock adjustment requested',
        $data
    );

    AuditLogger::log(
        (int) $user['id'],
        'INVENTORY',
        'REQUEST_STOCK_ADJUSTMENT',
        (int) $data['product_id'],
        null,
        $data
    );

    Response::success([
        'approval_id' => $approvalId,
    ], 'Stock adjustment request submitted for approval');
}
```

Then update `stockIn()` by adding this line at the top:

```
$user = Auth::requireRole([
    'SUPER_ADMIN',
    'ADMIN',
    'MANAGER',
    'STOREKEEPER'
]);
```

And after successful stock movement, before response, add:

```
AuditLogger::log(
    (int) $user['id'],
    'INVENTORY',
    'STOCK_IN',
    (int) $data['product_id'],
    null,
    $data
);
```

5H. Update

backend/app/Controllers/OrdersController.php

Add these imports:

```
use Support\Auth;
use Support\AuditLogger;
use Support\ApprovalEnforcer;
```

Add these methods inside the class:

```
public function requestCancellation(int $orderId): void
{
    $user = Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'WAITER',
        'CASHIER'
    ]);

    $data = Request::body();

    $approvalId = ApprovalEnforcer::createRequest(
        (int) $user['id'],
        'ORDER_CANCELLATION',
        $orderId,
        $data['reason'] ?? 'Order cancellation requested',
        $data
    );

    AuditLogger::log(
        (int) $user['id'],
        'ORDERS',
        'REQUEST_ORDER_CANCELLATION',
        $orderId,
        null,
        $data
    );

    Response::success([
        'approval_id' => $approvalId,
    ], 'Order cancellation request submitted for approval');
}

public function requestDiscount(int $orderId): void
{
    $user = Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'WAITER',
        'CASHIER'
    ]);

    $data = Request::body();

    $approvalId = ApprovalEnforcer::createRequest(
        (int) $user['id'],
        'DISCOUNT',
        $orderId,
        $data['reason'] ?? 'Discount requested',
        $data
    );

    AuditLogger::log(
        (int) $user['id'],
        'ORDERS',
        'REQUEST_DISCOUNT',
        $orderId,
        null,
        $data
    );

    Response::success([
        'approval_id' => $approvalId,
    ], 'Discount request submitted for approval');
}
```

Then update store() by adding this line at the top:

```
$user = Auth::requireRole([
    'SUPER_ADMIN',
    'ADMIN',
    'MANAGER',
    'WAITER',
    'CASHIER'
]);
```

And before the final response, add:

```
AuditLogger::log(
    (int) $user['id'],
    'ORDERS',
    'CREATE_ORDER',
    $orderId,
```

```
        null,  
        $data  
    );  
};
```

5I. Update

backend/app/Controllers/PaymentsController.php

Add these imports:

```
use Support\Auth;  
use Support\AuditLogger;
```

Add this line at the top of cashSettle():

```
$user = Auth::requireRole([  
    'SUPER_ADMIN',  
    'ADMIN',  
    'MANAGER',  
    'CASHIER'  
]);
```

Add this before the response:

```
AuditLogger::log(  
    (int) $user['id'],  
    'PAYMENTS',  
    'CASH_SETTLEMENT',  
    $orderId,  
    null,  
    $data  
);
```

5J.

docs/BACKEND_SECURITY_AUDIT_LAYER.md

Batch 5 – PHP Backend Security + Audit Layer

Purpose

This batch introduces backend control so the system does not merely record transactions, but also controls risk.

Security Added

- Auth user resolver
- Required authentication
- Role-based permissions
- Approval request system
- Audit logging
- Protected stock movement
- Protected order cancellation
- Protected discount requests
- Protected cash settlement

Approval Required For

1. Order cancellation
2. Discount
3. Stock adjustment
4. Refunds

Roles With Approval Power

- SUPER_ADMIN
- ADMIN
- MANAGER

Operational Protection

Stock Adjustment

Storekeepers may request adjustment, but management must approve before sensitive stock changes proceed.

Order Cancellation

Waiters and cashiers may request cancellation, but management must approve.

Discount

Waiters and cashiers may request discount, but management must approve.

Cash Settlement

Only cashier and management-level users may settle payments.

Audit Rule

Every sensitive action must write to `audit\_logs`.

Theft Control Impact

This layer helps prevent:

- fake stock adjustment
 - fake order cancellation
 - unauthorized discount
 - unauthorized cash settlement
 - untraceable system changes
-

Batch 5 Outcome

Backend now includes:

- ✓ auth middleware
- ✓ token validation
- ✓ role permissions
- ✓ audit logger
- ✓ approval enforcement
- ✓ stock adjustment protection
- ✓ cancellation approval protection
- ✓ discount approval protection

Next Smart Move

Build **Batch 6 — PHP Backend Operational Controllers**, including:

- bar controller
- kitchen controller
- dispatch controller
- unsold food controller
- asset controller
- HRM controller
- staff performance controller
- finance controller
- income statement controller